



程序设计语言初步

-关系运算和逻辑运算

软件与理论中心 张艳梅



二、关系运算符和关系表达式

- 关系运算符：用于比较两个数据的大小关系。

< 小于	<= 小于等于
> 大于	>= 大于等于
= 等于	!= 不等于

- 关系表达式：用关系运算符将两个表达式连接起来构成的表达式，如 $(7+3)<10$ ；运算结果是逻辑“真”-用1表示或者“假”-用0表示。
- 思考： $7+3<10$, 应如何解释？

关系运算的优先级

- 编程语言规定了运算符的优先级和结合性。

在表达式求值时，
算术运算高于关系运算，
小于、大于关系优先于相等和不等关系。

关系运算的结合性是左结合。

例：3>10/5-1,

a<b<c,

a==b<=c,

Operator	Name
!	Logical NOT. Bang.
++ --	Increment and decrement operators.
* / %	Multiplicative operators.
+ -	Additive operators.
<< >>	Shift operators.
< > <= >=	Inequality comparators.
== !=	Equality comparators
&	Bitwise AND.
^	Bitwise XOR.
	Bitwise OR.
&&	Logical AND.
	Logical OR.
?:	Conditional.
= op=	Assignment.

两个整数值相等是严格定义的，但计算机中浮点数类型表示的实数是有限精度，近似表示，因此应该避免对两个浮点数进行相等比较。

- 但是数学上，两小数的近似相等是有意义的，因此，两个浮点数只应作差的绝对值误差范围的比较，即 $a \approx b$ ，在计算机中应表达为 $|a-b| < \text{误差值}$ 。
- 例：判断3.1415926和三角函数计算值之间误差是否在小数点后6位。

`fabs(3.1415926-acos(-1)) < 1e-6`

语法:

```
#include <math.h>
```

```
double fabs( double arg );
```

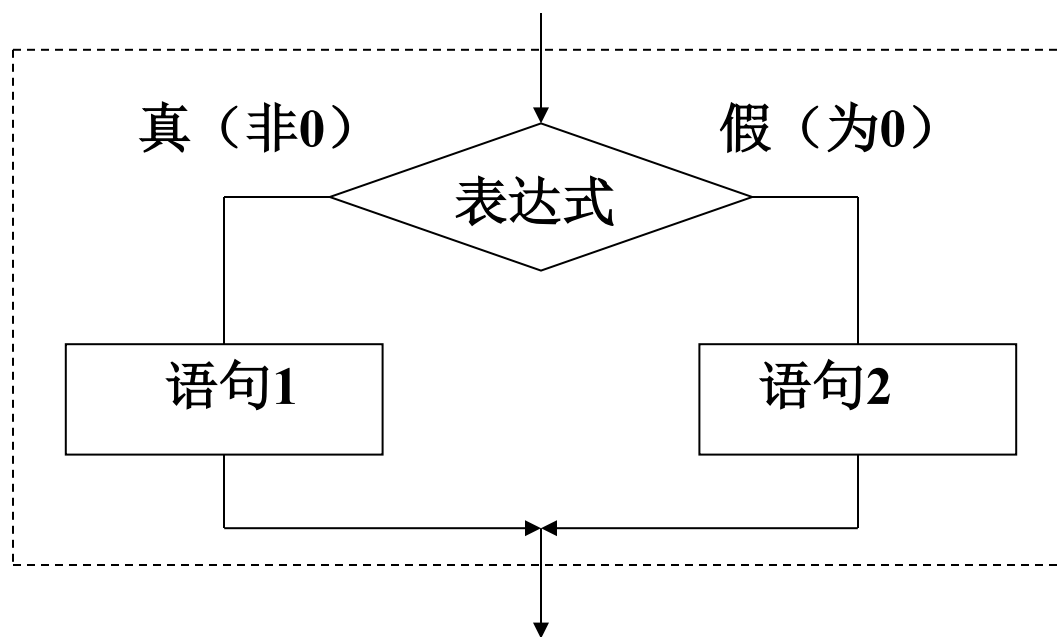
功能: 函数返回参数arg的绝对值。

```
#include<stdio.h>
#include<math.h>
int main(void){
    int a;
    a=7+3<10;
    printf("7+3<10  =%d\n",a);
    printf("3>10/5-1  =%d\n",3>10/5-1);
    printf("3<2<1  =%d\n",3<2<1);
    printf("1==2<=3  =%d\n",1==2<=3);
    a=fabs(3.1415926-acos(-1))<=1e-6;
    printf("pai difference judge is %d\n",a);
}
```

是非选择结构

if-else

实现逻辑判断



a) 流程图

```
if (表达式)
    语句1
else
    语句2
```

b) C语法

使用大括号构造复合语句

自然语言中，简单表达只用一句话，复杂表述用段落。C语言规定，单条语句用;结束，复合语句由大括号和语句段组成。

```
例:  if (n >3)
      {
          i = 1;
          j = 2 * i;
      }
      else
      {
          i = 0;          j = 2*i+1;
      }
```


三、逻辑运算符和逻辑表达式

逻辑运算符：与and、或or、非not

&& 逻辑与	a&&b: 当且仅当操作数a和b的值都为“真”或非0时, 运算结果为“真1”, 否则为“假0”
逻辑或	a b: 当且仅当操作数a和b值都为“假”或0时, 运算结果为“假0”, 否则为“真1”。
! 逻辑非	!a: 当操作数a的值为“真”或非0时, 运算结果为“假0”; 当操作数a的值为“假”或0时, 运算结果为“真1”。

注意：逻辑运算和关系运算、算术运算可以混合在一起运算。

自然语言中的或语义

- 在自然语言中 “或 or” 有两种不同的含义。
 - “兼容或” - 陈述句 “计算机专业或数学专业的学生可以上离散数学课,” 其含义是假设学生必须满足前提中的一种, 当然两种都是更没问题. 所以这里的含义和逻辑或是是一致的.
 - “排他或/异或” - 当我们读到 “饭后甜点可选咖啡或红茶” 这句话时, 意思是咖啡和红茶只能二选一。这里的 或 是排他或(XOR)的含义。在两个选项中, 只能有一个是真的, 但不能两者都选。

逻辑运算的优先级

- 编程语言规定了运算符的优先级和结合性。

在表达式求值时，
逻辑非运算最高，
然后是算术运算和关系运算，
逻辑运算的优先级仅高于赋值运算。

逻辑运算的结合性默认是左结合，只有非运算是右结合。

Operator	Name
!	Logical NOT. Bang.
++ --	Increment and decrement operators.
* / %	Multiplicative operators.
+ -	Additive operators.
<< >>	Shift operators.
< > <= >=	Inequality comparators.
== !=	Equality comparators
&	Bitwise AND.
^	Bitwise XOR.
	Bitwise OR.
&&	Logical AND.
	Logical OR.
?:	Conditional.
= op=	Assignment.

奥古斯塔●德●摩根
(1806-1871)

- 逻辑运算中与关系的否定运算律是
 - $\text{Not}(p \text{ and } q)$ 等价于 $\text{not } p \text{ or not } q$
- 逻辑运算中或关系的否定运算律是
 - $\text{Not}(p \text{ or } q)$ 等价于 $\text{not } p \text{ and not } q$
 - 用真值表证明德摩根第二定律:

p	q	$\text{not } p$	$\text{not } q$	$(p \text{ or } q)$	$\text{not } (p \text{ or } q)$	$\text{not } p \text{ and not } q$
T	T	F	F	T	F	F
T	F	F	T	T	F	F
F	T	T	F	T	F	F
F	F	T	T	F	T	T

- 写出下列表达式的否定形式:

1. $1 == 3/2 \ \&\& \ 3 > 2$

2. $5/2 > 2 \ || \ 1$

3. $!(2 < 3) \ || \ (3 \ \&\& \ 5)$

4. $!a \ \&\& \ b \ || \ c$

- 已知边长 a, b, c ,
 - 判断是否构成三角形。
 - 判断是否直角三角形。
 - 判断是否等边三角形。
 - 判断 a 和 b 是相邻整数。反向判断呢？
 - 判断 a, b, c 是等差数列。反向判断呢？

问题：解方程

问题描述：求 $ax^2+bx+c=0$ 的根。若 a 为零，则不是二次方程不予计算；若 a 不为零，分别考虑 $d=b^2-4ac$ 大于零，等于零和小于零三种情况。

a 、 b 、 c 要求是浮点单精度类型。

Sample 1:

Please input three float numbers: 2.1 -9 8✓

The equation has two distinct real roots: 3.0273 and 1.2584.

Sample 2:

Please input three float numbers: 3.5 3.6 8.0✓

The equation has two complex roots: $-0.5143+1.4217i$ and $-0.5143-1.4217i$.

解方程的算法设计:

Begin

定义浮点数变量a, b, c, d, x1, x2;

从键盘接受输入的实数, 存入a, b, c;

if a等于零, 用 a的绝对值小于 $1e-6$ 来判断

显示器打印非二次方程的提示信息;

else 计算 $d=b^2-4ac$,

if d等于零, 用 d的绝对值小于 $1e-6$ 来判断

显示器打印重根 $x1=x2=-b/(2*a)$;

else if d大于零, 用 $d>1e-6$ 来判断

$x1=(-b+\text{sqrt}(d))/(2*a)$;

$x2=(-b-\text{sqrt}(d))/(2*a)$;

显示器打印根x1和x2;

else 一定是d小于零

$x1=-b/(2*a)$; $x2=\text{sqrt}(-d)/(2*a)$;

显示器打印"%8.4f±%8.4fi", x1, x2;

End

编写解方程的代码

```
#include <stdio.h>
#include <math.h>
#define DIFF 1e-6
int main(void){
    float a,b,c; /*ax2+bx+c=0 中的a,b,c*/
    float d; /*d=b*b-4*a*c*/
    float x1,x2; /*实根,或虚根的实数和复数部分*/
    /*input a、 b、 c*/
    printf("Please input three float numbers: ");
    scanf("%f%f%f",&a,&b,&c);
    /*process*/
    printf("The equation ");
    if (fabs(a)<DIFF)
        printf("is not quadratic.\n");
```

编写解方程的代码

```
else{
    d=b*b-4*a*c;
    if(fabs(d)<DIFF)
        printf("has two equal roots: %.4f.\n",-
b/(2*a));
    else if(d>DIFF){
        x1=(-b+sqrt(d))/(2*a);
        x2=(-b-sqrt(d))/(2*a);
        if (x1>x2)
printf("has two distinct real roots: %.4f and
%.4f.\n",x1,x2);
        else
printf("has two distinct real roots: %.4f and
%.4f.\n",x2,x1);
    }
}
```

```
else{
    x1=-b/(2*a);
    x2=sqrt(-d)/(2*a);
    if (a>0)
        printf("has two complex roots: %.4f+%.4fi
and %.4f-%.4fi.\n",x1,x2,x1,x2);
    else
        printf("has two complex roots: %.4f+%.4fi
and %.4f-%.4fi.\n",x1,-x2,x1,-x2);
    }
}
return 0;
}
```

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1: 0.4 1.2 0.9; 重根-1.5000.

测试数据2: -2 -8 -2; 根-0.2679, -3.7321.

测试数据3: 1 2 3; 复根-1.0000±1.4142i.

测试数据4: 0 2 3 ; 非二次方程.

测试数据5: -509.86 489.60 689.16; 根
1.7380, -0.7777.

- 发生意外或错误，程序无法继续处理时，需要终止，操作系统提供了**exit()**函数。

- 函数名：**void exit(int status);**

//参 数**status**，程序退出的返回值

所在头文件：**stdlib.h**

功 能：关闭所有文件，终止正在执行的进程。

exit(0)表示正常退出，

exit(x)（**x**不为**0**）都表示异常退出，这个**x**是返回给操作系统（包括**UNIX, Linux**, 和**MS DOS**）的，以供其他程序使用。



```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    printf("Start of the program....\n");

    printf("Exiting the program....\n");
    exit(0);

    printf("End of the program....\n");
    return(0);
}
```



解方程代码添加**exit**后，结构较清晰

```
/*process*/
if(fabs(a)<DIFF)
{
    printf("is not quadratic.\n");
    exit(1);
}
d=b*b-4*a*c;
if(fabs(d)<DIFF)
    printf("has two equal roots: %.4f.\n",-
b/(2*a));
else if(d>DIFF){ ...
}
else{ ...
}
...
```

取余运算的用途1：整除判断

取余运算的本质是计算两个整数不能整除的余数。因此，余数为0可以作为整除关系的判断条件；余数不为0可以作为不能整除的各类情况判断。

1. 定义程序目标

问题：输入任意一个整数 n ，程序应输出 n 是奇数还是偶数的判断结果。

说明：明确程序需要哪些信息，进行什么计算，报告什么结果。

2. 设计程序

问题：程序用户是谁？

问题：有多久的开发时间？

问题：程序界面如何？

最简单的字符界面和键盘交互。

问题：如何表示数据？

整数。

问题：如何处理数据？

偶数是2的倍数，即除2余0；可用取余运算。

整数奇偶判断的算法设计：

Begin

 定义一个整数变量n；

 从键盘接受输入的一个整数，存入n；

 if ($n \% 2 == 0$)

 显示器输出打印“n是偶数”；

 else

 显示器输出打印“n是奇数”；

End

3. 编写代码

首先：选择编程语言。

然后：根据算法，对应所选编程语言的编码规范；

算法需要整数变量表示和整数取余运算，选择语句，以及输入、输出操作。

编程活动步骤3-编写代码

整数奇偶判断的算法设计:

Begin

 定义一个整数变量n;

 从键盘接受输入的一个
整数, 存入n;

 if (n%2==0)

 显示器输出打印

 “n是偶数”;

 else

 显示器输出打印

 “n是奇数”;

End

```
/*在程序员自定义内容区域敲代码  
*/
```

```
    int n;  
    printf("Please input a  
integer :\n");  
    scanf("%d",&n);  
    if(n%2==0)  
        printf("%d is  
even.\n",n);  
    else  
        printf("%d is  
odd.\n",n);
```

4. 编译

目的：检查程序有效性（符合编程语言语法）。

5. 测试和调试程序

用测试数据运行程序，查看结果是否有错。

测试数据1: 0;

测试数据2: 3;

测试数据3: 12;

测试数据4: -8;

测试数据5: -3;

- 问题：输入四位数的年份，判定该年有多少天？
- 算法设计：
 - 每年的天数只有两种情况，闰年366天，非闰年365天。
 - 闰年如何判断？4年一闰，100年不闰，400加一闰。
 - 三个周期的整除关系组合判断。

判断一年的天数（闰年）

```
#include <stdio.h>
int main(void){
    int year;

    printf("Enter the year :");
    scanf("%d",&year);
    if ((year%4==0 && year%100!=0) || (year%400==0))
        printf("%d year has 366 days.\n",year);
    else
        printf("%d year has 365 days.\n",year);

    return 0;
}
```

问题：字母的序号和大小写转换

输入一个字符，判断是否为英文字母，若是，给出在字母表中的序号(1~26); 再将大写变为小写，小写转为大写。

样例输入： C

样例输出： Yes 3 c

字母判定与大小写转换

字母判断与大小写转换算法设计：

Begin

定义字符变量letter;

从键盘接受输入的一个字符，存入letter;

if(letter是大写字母 或 letter是小写字母)

显示器输出打印Yes;

if(letter是大写字母)

显示器输出打印序号(letter- 'A');

显示器输出打印对应小写字母letter+32;

else if(letter是小写字母)

显示器输出打印序号(letter- 'a');

显示器输出打印对应大写字母letter-32;

else 显示器输出打印No;

End

字母大小写转换程序

```
#include <stdio.h>
int main(void){
    char ch;
    printf("Enter a character: ");
    ch = getchar();

    if (ch >= 'A' && ch <= 'Z') {
        ch += 32; //等效于ch += 'a' - 'A';
        printf("Yes %d %c\n", ch - 'a' + 1, ch);
    }
    else if (ch >= 'a' && ch <= 'z'){
        ch -= 32; //等效于ch -= 'a' - 'A';
        printf("Yes %d %c\n", ch - 'A' + 1, ch);
    }
    else
        printf("No\n");

    return 0;
}
```

4. 编译

目的：检查程序有效性（符合编程语言语法）。

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1: Q;

测试数据2: Z;

测试数据3: a;

测试数据2: m;

测试数据5: 3;

测试数据6: #;

测试数据7: Hi!;

问题：识别数码

输入一个字符，判断是否为阿拉伯数字符号。若是，则输出**Yes**和对应的八进制数值；否则输出**No**。

样例输入： 8

样例输出： **Yes, 10.**

样例输入： b

样例输出： **No**

字符相邻的算法设计：

Begin

定义字符变量letter;

从键盘接受输入的字符，存入letter;

if(letter在数码'0'到'9'的范围内)

显示器输出打印Yes;

定义整数变量num;

计算letter的数值存入num;

显示器按8进制格式输出打印num;

else 显示器输出打印No;

End

完善：数码letter的数值计算表达式。

(letter - '0')

编写识别数码的程序

```
#include <stdio.h>
int main(void){
    char letter;
    int num;

    printf("Enter a digit character: ");
    letter = getchar();

    if (letter >= '0' && letter <= '9')
    {
        num = letter - '0';
        printf("Yes,%o.\n", num);
    }
    else
        printf("No\n");

    return 0;
}
```

4. 编译

目的：检查程序有效性（符合编程语言语法）。

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1： 0；

测试数据2： 8；

测试数据3： 9；

测试数据2： A；

测试数据5： w；

测试数据6： #；

测试数据7： 19；

问题：字符相邻判断

输入两个字符，判断是否在ASCII编码表中相邻。
若相邻，输出Yes;否则输出No。

样例输入： hi

样例输出： Yes

样例输入： Cb

样例输出： No

字符相邻的算法设计：

Begin

定义字符变量letterA和letterB;

从键盘接受输入的两个字符，存入letterA和letterB;

if (letterA和letterB的数值仅差1)

显示器输出打印Yes;

else 显示器输出打印No;

End

完善：字符值仅差1的表达式书写。

情况1, $\text{letterA} - \text{letterB} == 1$

情况2, $\text{letterB} - \text{letterA} == 1$

两种情况是 或 关系。

```
#include <stdio.h>
int main(void){
    char letterA,letterB;
    printf("Enter two character: ");
    letterA = getchar();
    letterB = getchar();

    if ((letterA - letterB == 1) || (letterB -
letterA == 1))
    {
        printf("Yes\n");
    }
    else
        printf("No\n");

    return 0;
}
```

4. 编译

目的：检查程序有效性（符合编程语言语法）。

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1: hi; 测试数据2: po;

测试数据3: AB; 测试数据2: ZA;

测试数据5: 32; 测试数据6: 01;

测试数据7: Hi; 测试数据6: U8;

思考：

如果要判断连续xyz三个字符是否前后相邻呢？

```
if (abs(x-y)==1 && abs(y-z)==1)
```

```
if ((x-y==1 || x-y==-1)&&(y-z==1 || y-z==-1))
```

